

Module 2 — Workflows & Best Practices

Lesson 2.6

When to delegate to a subagent

Delegate work whose noise you don't want in your main context, or whose perspective you want independent of your own. Inline everything else.

Mastering Claude Code — An E-Course for Power Users

Why it matters

Subagents are the most powerful, most over-used feature in Claude Code. Used well, they protect your context and parallelize work. Used badly, they multiply latency and produce duplicate effort. The decision rule matters.

The decision rule

Delegate to a subagent when **at least one** of these is true:

- 1 The task will read or generate a lot of context Claude doesn't need to retain (search across many files, parse large logs, scan dependencies).
- 2 You want an **independent perspective** — a fresh read uncorrupted by your reasoning so far.
- 3 The work is **parallelizable** — multiple independent units of work that can run concurrently.

Do *not* delegate when:

- The task is small (one file, one function, one query) — overhead exceeds benefit.
- The task depends on context only you have in the current session — the subagent will start cold.
- The task requires multiple back-and-forth turns with you — subagents don't loop with the user; they get one brief and return.

The four common patterns

Pattern 1 — Search-and-report

Delegate when you want answers, not raw search output.

```
"Use the Explore subagent. Brief: find every place we still reference the
old `legacyApiClient`. Return a list of file:line locations and the
context of each call. Under 200 words."
```

Cost to your context: ~200 words. Cost without delegation: thousands of tokens of grep output.

Pattern 2 — Independent review

Delegate when you want a fresh opinion. The subagent has *no* memory of how you got here.

```
"Spawn a code-reviewer subagent. Brief: review the diff in `git diff main`.
I've been deep in this for two hours; I want an independent read.
Focus areas: correctness of the new SQL, security of the auth handling.
Report top issues only – assume I'll iterate on the rest later."
```

The independence is the value. If you brief it with your conclusions, you've poisoned the well.

Pattern 3 — Parallel fan-out

Delegate when you have N independent tasks. Spawn N agents in one message.

```
[message with three Agent tool calls in the same block]:
Agent 1: "Migrate src/billing/ from Mocha to Vitest. Report file count."
Agent 2: "Migrate src/auth/ from Mocha to Vitest. Report file count."
Agent 3: "Migrate src/users/ from Mocha to Vitest. Report file count."
```

All three run concurrently. Reduces wall-clock time roughly 3x for the parallelizable portion.

Cap: 3 concurrent agents is the practical ceiling. More and you'll hit rate limits or context-thrash.

Pattern 4 — Isolated experiment via worktree

Delegate when you want changes attempted on a parallel copy of the repo without polluting your working tree.

```
"Spawn a general-purpose agent with isolation: worktree. Brief: try
implementing the migration with approach B (synchronous backfill).
Report whether it works and what trade-offs it surfaced."
```

The agent works in a temp worktree; you can compare without merging anything you don't want. We cover this in 4.5 Worktrees.

Anti-patterns

Anti-pattern	Why it's bad
Spawning an agent to "do the task for me"	If you don't know the task well enough to brief, you don't know it well enough to delegate. Inline it and learn.
Spawning an agent for every step	Latency adds up; you also lose the agent's findings between calls.
Briefing an agent with "based on your earlier findings..."	New agent invocations have no memory of prior runs. Always self-contained.
Delegating then duplicating	If you delegated the search, don't also grep yourself. Wait for the report.

Sequential vs. parallel: a checklist

Use parallel when:

- The tasks don't share state
- You can write the briefs without knowing the others' outputs
- You'll merge results, not chain them

Use sequential when:

- Task B's brief depends on task A's findings
- You need to course-correct between steps

Try it

- 1 Pick a task involving search across many files. Do it once inline — note how much context it eats.
- 2 Do an equivalent task by spawning an Explore agent with a tight brief. Compare context usage and felt speed.

- 3 Try a 3-way fan-out for something genuinely parallel (e.g., three independent file migrations). Note the wall-clock difference.

Gotchas

- **Trust but verify.** A subagent's report says what it intended, not necessarily what it did. If it wrote code, read the diff.
- **Don't ask for a "comprehensive report" from a subagent** unless you actually need 2000 words. Ask for a punch list. Cap word count.
- **Subagents can't ask you questions in the middle.** All clarification has to happen before the agent starts. If your brief is ambiguous, you'll get a surprising result.
- **Some agent types are restricted to read-only tools** (e.g., Explore). Don't expect them to write code.

Further reading

- Module 4 Subagents — the whole module
- 4.4 Parallel orchestration — the fan-out pattern in depth

Next

→ 3.1 CLAUDE.md — project memory done right